Lagom-style service descriptor for announce-management

Session report for worktree `service-descriptor` (branch `feat/service-descriptor`), 2026-10-24. The plan (`doc/service-descriptor-plan.pdf`, commit ca9129bc) was approved and implemented as written; this report records what exists now, how it was verified and what was left out.

1. What changed, in one paragraph

The read slice of the announce API is now a service descriptor: `AnnounceApi` in `announce-management-api` declares `view` and `browse` with the mappings and the response records, `AnnounceController` implements it (Spring Framework 6.2 maps the interface server-side, so the route is declared once), the caller's identity moved out of the two method signatures into `CallerContextFilter` plus the Reactor context, messaging dropped its private copy of the descriptor and compiles against the api module, and the framework sample shows the same pattern with a client round trip under the service token. The relay remains the only Kafka producer and `cqrs-reactive` main code is untouched.

2. The pieces

Module	Piece	What it does
announce-management-api	`AnnounceApi`	`@HttpExchange("/api/announces")` with `@GetExchange("/{id}") view` and `@GetExchange browse` (every filter `@RequestParam(required = false)`, page 1, size 20, sort newest). Parameters are client-legal only and named explicitly.
announce-management-api	`AnnounceApiDtos`	Logic-free records: AnnounceResponse, AnnounceSummaryResponse, PageResponse, LocationDto, GoodsDto, JobDto, EventDto, VariantDto, AttributeDto, SpecificationDto. Same field names and JSON as before.
announce-management-api	V1 enums	AnnounceKindV1, AnnounceStatusV1, SaleTypeV1, ConditionV1, ContractTypeV1: same constant names as the domain enums, serialised as strings.
announce-management	`AnnounceApiMapper`	The only class that knows both sides: request to AnnounceContent / AnnounceKind, Announce to AnnounceResponse, AnnounceSummary to AnnounceSummaryResponse, enums by name both ways.
announce-management	`AnnounceDtos`	Shrunk to the request bodies AnnounceRequest (delegates to the mapper) and PublishRequest.
announce-management	`CallerContextFilter`	WebFilter at MIN_VALUE + 300, after ServiceTokenWebFilter. Resolves X-User-Id / X-User-Type once into the Reactor context as a Callers.Resolution (caller, nobody, or the stored parse failure).
announce-management	`Callers`	New no-arg overloads member(), viewer(), staff(permission) returning Mono from the context; the HttpHeaders overloads stay. Same problem details as before (401 no-principal, 403 not-a-member / not-staff / missing-permission, 400 for a non-numeric id).
announce-management	`AnnounceController`	implements AnnounceApi; no class @RequestMapping, no @GetMapping on view / browse; writes unchanged with @PostMapping / @PutMapping and HttpHeaders.
announce-management	Admin and brand controllers	Answer with the api records through the mapper (AnnounceAdminController) or the api PageResponse (BrandController).
messaging	`AnnounceLookup`	Nested record AnnounceFacts(id, sellerId, title, status) replaces AnnounceRef; AnnounceLookupConfig creates the shared AnnounceApi client and maps AnnounceResponse into it; 404 stays an empty Mono. pom gains announce-management-api. Private AnnounceApi and AnnounceRef deleted.
cqrs-reactive-test	`ListingApi`, `ListingController`	Sample descriptor (post with optional X-Command-Id header, view) implemented by the controller; view answers a 404 problem detail listing-not-found instead of an empty ResponseEntity. SampleEndToEndTest runs with the service token enforced and adds the client round trip.

Module	Piece	What it does
doc	`cqrs-reactive.md`	§4 "Implementing the descriptor" (rules below) and a new Lagom table row "service implementation implements the descriptor".
doc	`announce-management-context.md`	§12: the api module layout, the controller, mapper and filter, what messaging keeps of a response.

3. Rules now written down in doc/cqrs-reactive.md §4

- Never mix the two annotation families on one element: a class with @RequestMapping must not inherit @HttpExchange, a @GetMapping method must not override a @GetExchange method. Mixing across methods is fine; the interface's class-level path combines with a method-level @PostMapping.
- Only client-legal parameters on the interface: @PathVariable, @RequestParam, @RequestBody, named @RequestHeader. Name every path variable and request parameter explicitly.
- Identity is not a parameter: the edge forwards it, a WebFilter resolves it into the Reactor context, routes read it from there. A service calling through HttpExchangeClients sends no identity and is served as a visitor (PUBLISHED announces only).
- DTOs on the interface live in the -api module without logic and with api-level enums; the provider keeps a mapper. Add fields, never rename or retype.
- @ResponseStatus stays on the implementation; 4xx answers reach the client as WebClientResponseException subclasses.
- The write path never produces to Kafka; the PublicEventRelay reading the journal is the only producer.

4. Deviations from the block and the plan

- Two extra api enums, ConditionV1 and ContractTypeV1, because the response records carry those domain enums too and the api module may not import the domain (plan §3, reading 1).
- AnnounceRef is replaced by the nested record AnnounceLookup.AnnounceFacts rather than by AnnounceResponse itself, keeping messaging's application layer off the wire type (plan §3, reading 2).
- The sample's view answers a 404 problem detail (CommandRejectedException, code listing-not-found) instead of a ResponseEntity, because a client proxy turns a 404 into an exception anyway and the descriptor reads better with a plain Mono.
- The sample test keeps ListingSummaryProjection.Summary as the view record on the descriptor; in a real service that record would live in the -api module (said in the ListingApi javadoc).

5. Verification

Suite	Command	Result
framework + test kit	`mvn -pl cqrs-reactive,cqrs-reactive-test test`	BUILD SUCCESS; cqrs-reactive-test 20 tests, SampleEndToEndTest 7 (new round trip included); Testcontainers only
announce-management	`mvn -pl cqrs-reactive,announce-management-api,announce-management -P '!webapp' test`	BUILD SUCCESS; 75 tests, 0 failures: AnnounceKindsEndToEndTest 8, AnnounceApiRoundTripTest 4, AnnounceControllerTest 8, AnnounceAdminControllerTest 5, MyBatisRepositoriesTest 1 (live MySQL)
messaging	`mvn -pl cqrs-reactive,announce-management-api,messaging test`	BUILD SUCCESS; 50 tests, 0 failures, 1 skipped (StatementCostProbeTest runs only on request); MessagingEndToEndTest 13 on the stub lookup

All three runs on 2026-10-24 between 14:22 and 14:26, built in one reactor with cqrs-reactive and announce-management-api so no stale jar from the shared ~/.m2 could leak in. Docker was up for Testcontainers (EventSourcingDB, Kafka, MySQL); the live announce_reactive_db served the repository tests.

Acceptance checks of the block: messaging has no local descriptor and compiles against announce-management-api; AnnounceApiRoundTripTest calls view and browse through HttpExchangeClients (base URL pinned, service token on) and gets the same JSON the controller serves to a signed direct request, a client without the token gets 401; AnnounceControllerTest shows a request without X-User-Id sees PUBLISHED announces only and the owner sees their drafts, over the caller filter; the sample round trip in cqrs-reactive-test passes with the service token enforced; §4 of the framework doc is updated. The manual check with the stack up (member opens a listing conversation, messaging log shows the call through the shared descriptor) was run on 2026-10-24 with the stack up and recorded in doc/frontend-edges.md §7b: a new member (id 61) opened a listing conversation on announce 2000001 both over REST and by clicking "Message the seller" in the customer SPA (headless Chromium); messaging logged "HTTP GET http://localhost:8085/api/announces/2000001" and "Response 200 OK" for each open, the seller (1) was resolved through the shared descriptor, an unknown announce logged a 404 and answered the problem detail announce-not-found.

6. How to put the next route on the descriptor

- Add the method to the interface in the -api module with client-legal, explicitly named parameters; move or create its records there with V1 enums.
- Remove the mapping annotation from the controller method and mark it @Override; keep @ResponseStatus if any. Read the caller with Callers.member() / viewer() / staff(permission) from the context.
- Extend the mapper for new record shapes; extend AnnounceApiRoundTripTest with the new round trip.
- Consumers replace their private interface with the shared one; the client is unchanged (HttpExchangeClients.create).

7. Left out, on purpose

Member writes on the interface (post, edit, publish, withdraw, sold), the admin and listing controllers on the caller filter, descriptors for order and messaging, DTO versioning beyond the V1 enums, any change in cqrs-reactive main code.